

Alias en SQL

¿Qué es un alias?

En SQL, un alias (o "alias" en español) es un nombre alternativo para una tabla o columna que se utiliza dentro de una consulta SQL. Permite referirse a la tabla o columna usando un nombre más corto o descriptivo, lo que puede mejorar la legibilidad y la claridad de las consultas.

En resumen, un alias:

- **Es un nombre temporal:** Solo existe durante la ejecución de la consulta.
- **Facilita la lectura:** Simplifica la consulta y hace que sea más fácil de entender.
- **Permite referirse a tablas o columnas:** Tanto para simplificar nombres largos como para crear nombres más descriptivos.
- **No afecta la base de datos:** El nombre real de la tabla o columna en la base de datos no cambia.

Ejemplos:

1. Alias de tabla:

Code

```
SELECT * FROM productos AS p;
```

En este ejemplo, la tabla productos se referencia a través del alias p. Esto facilita la escritura de nombres calificados (ej: p.nombre) en consultas que involucran múltiples tablas.

1. Alias de columna:

Code

```
SELECT nombre AS nombre_completo FROM clientes;
```

Aquí, la columna nombre se referencia como nombre_completo dentro del resultado de la consulta. Esto permite mostrar el nombre de la columna de una forma más descriptiva.

1. Alias en JOINs:

Code

```
SELECT p.nombre, c.categoria  
FROM productos AS p  
JOIN categorias AS c ON p.id_categoria = c.id_categoria;
```

En este ejemplo, los alias p y c se utilizan para referirse a las tablas productos y categorias dentro de una consulta JOIN. Esto mejora la legibilidad al referir a columnas de ambas tablas usando p.nombre y c.categoria.

Usos adicionales:

- **Simplificar consultas complejas:**

Los alias pueden hacer que las consultas con múltiples tablas y joins sean más legibles y mantenibles.

- **Ocultar nombres de tablas o columnas sensibles:**

Los alias pueden ayudar a proteger nombres de tablas o columnas que puedan ser sensibles o no deseados en la consulta final.

- **Crear resultados significativos:**

Cuando se usan funciones de agregación, los alias pueden ayudar a identificar el resultado de la función de manera más descriptiva.

GROUP BY y ORDER BY

Cuando trabajamos con grandes conjuntos de datos en SQL, frecuentemente necesitamos resumir y organizar la información en tablas más manejables. Es aquí donde entran en juego las cláusulas GROUP BY, ORDER BY y HAVING.

La cláusula GROUP BY nos permite agrupar filas en una tabla según los valores en una o más columnas. La cláusula ORDER BY permite ordenar las filas de una tabla según los valores en una o más columnas. Y la cláusula HAVING filtrar grupos de filas según el resultado de una función de agregación.

Usaremos una tabla de ventas que contiene información sobre los pedidos realizados por los clientes en una tienda en línea. La tabla se llama ventas y se ve:

id_venta	fecha_venta	id_cliente	producto	cantidad	precio_unitario
1	2023-05-01	101	producto_A	3	10
2	2023-05-02	102	producto_B	2	20
3	2023-05-03	103	producto_C	1	30
4	2023-05-04	101	producto_B	5	20
5	2023-05-05	102	producto_A	2	10
6	2023-05-06	104	producto_D	1	40
7	2023-05-07	101	producto_C	4	30
8	2023-05-08	102	producto_B	3	20
9	2023-05-09	103	producto_A	2	10
10	2023-05-10	101	producto_D	1	40

En esta tabla, cada fila representa una venta individual con su id, fecha de venta, id de cliente, producto vendido, cantidad vendida y precio unitario.

GROUP BY

GROUP BY es un comando de SQL que se utiliza para agrupar filas en una tabla en función del valor de una columna determinada. Por ejemplo, si tienes una tabla de ventas que incluye información sobre los productos vendidos, el precio y la fecha de venta, puedes utilizar GROUP BY para agrupar las ventas por producto y obtener el total de ventas por producto.

Se usa junto con funciones de agregación como SUM, COUNT o AVG para resumir los datos en cada grupo.

Sintaxis:

```
SELECT <columna1>, <columna2>, ..., <funcion_agregacion>(<columnaN>)  
FROM <tabla>  
GROUP BY <columna1>, <columna2>, ..., <columnaN>;
```

Si realizamos la siguiente consulta

```
SELECT producto, SUM(cantidad) as cantidad_total  
FROM ventas  
GROUP BY producto;
```

Este es el resultado:

producto	cantidad_total
producto_A	7
producto_B	10
producto_C	5
producto_D	2

Podemos ver que la cláusula GROUP BY agrupó todas las filas que tenían el mismo producto y luego sumó la cantidad vendida para cada producto.

ORDER BY

La cláusula ORDER BY se utiliza para ordenar las filas de una tabla según los valores en una o más columnas. Podemos ordenar en orden ascendente o descendente utilizando las palabras clave ASC o DESC.

```
SELECT <columna1>, <columna2>, ...  
FROM <tabla>  
ORDER BY <columna1> [ASC|DESC], <columna2> [ASC|DESC], ...;
```

Aquí columna1, columna2, ... son las columnas que se utilizan para ordenar las filas y [ASC|DESC] es opcional y especifica el orden en que se deben ordenar las filas.

Supongamos ahora la siguiente consulta, con nuestra tabla ejemplo:

```
SELECT id_venta, fecha_venta, id_cliente, producto, cantidad, precio_unitario  
FROM ventas  
ORDER BY fecha_venta ASC;
```

Ordenamos todas las ventas por fecha de venta en orden ascendente.

id_venta	fecha_venta	id_cliente	producto	cantidad	precio_unitario
1	2023-05-01	101	producto_A	3	10
2	2023-05-02	102	producto_B	2	20
3	2023-05-03	103	producto_C	1	30
4	2023-05-04	101	producto_B	5	20
5	2023-05-05	102	producto_A	2	10
6	2023-05-06	104	producto_D	1	40
7	2023-05-07	101	producto_C	4	30
8	2023-05-08	102	producto_B	3	20
9	2023-05-09	103	producto_A	2	10
10	2023-05-10	101	producto_D	1	40

HAVING

La cláusula HAVING se utiliza para filtrar grupos de filas según el resultado de una función de agregación. Permite filtrar grupos basados en una condición.

Sintaxis

```
SELECT <columna1>, <columna2>, ..., <funcion_agregacion>(<columnaN>)  
FROM <tabla>  
GROUP BY <columna1>, <columna2>, ..., <columnaN>  
HAVING <condicion>;
```

Ejemplo:

```
SELECT producto, SUM(cantidad) as cantidad_total  
FROM ventas  
GROUP BY producto  
HAVING SUM(cantidad) > 10;
```

Agrupamos todas las ventas por producto y luego filtramos los grupos que tengan una cantidad total vendida mayor que 10.

producto	cantidad_total
producto_A	12
producto_B	15
producto_C	9

Podemos apreciar que solo se muestran los grupos de ventas que tienen una cantidad total vendida mayor que 10.

¿Qué son las funciones agregadas en SQL y cómo se utilizan?

Las funciones de agregación en SQL son herramientas que permiten realizar cálculos sobre un conjunto de valores de una o más columnas y devolver un único valor como resultado. Son esenciales para resumir y analizar datos de manera efectiva.

Funciones comunes:

Función	¿Qué hace?	Ejemplo de uso
COUNT()	Cuenta cuántas filas hay	¿Cuántos pedidos hay?
SUM()	Suma los valores de una columna numérica	¿Cuánto dinero se ha ingresado por ventas?
AVG()	Calcula la media de los valores numéricos	¿Cuál es el precio medio de los productos?
MAX()	Devuelve el valor más alto de una columna	¿Cuál es el producto más caro vendido?
MIN()	Devuelve el valor más bajo de una columna	¿Cuál es la fecha del primer pedido?

Uso

Las funciones de agregación se utilizan típicamente con la cláusula SELECT en una consulta SQL. También se pueden combinar con la cláusula GROUP BY para agrupar los datos antes de aplicar la función de agregación

Por ejemplo,

```
SELECT AVG(precio)
FROM productos;
```

calcula el precio promedio de todos los productos.

Con la cláusula GROUP BY:

```
SELECT departamento, AVG(salario)
FROM empleados
GROUP BY departamento;
```

calcularía el salario promedio para cada departamento.

Otros ejemplos:

```
SELECT COUNT(*)  
FROM empleados;
```

Devuelve el número total de empleados.

```
SELECT MAX(precio)  
FROM productos;
```

Devuelve el precio más alto de todos los productos.

```
SELECT MIN(fecha_nacimiento)  
FROM empleados;
```

Devuelve la fecha de nacimiento más temprana de todos los empleados.